

ALFIDO TECH

Machine Learning Project Report

Customer Churn Classification

Predicting Customer Attrition Using Machine Learning

Prepared by

Rashmi C S

Organization

Alfido Tech

Repository

github.com/Rashmics96-dotcom/alfido-tech-task1-ml

August 2026

Table of Contents

1. Executive Summary..... 3

2. Introduction..... 3

3. Dataset Overview 4

4. Methodology 4

5. Results..... 6

6. Model Testing and Interactive Prediction 7

7. Key Findings and Limitations 8

8. Conclusion 9

9. Appendix 9

1. Executive Summary

This report presents the design, implementation, and evaluation of a machine learning system built to predict customer churn for a telecom-style subscriber base. The project was undertaken as part of the Alfredo Tech Machine Learning internship task and covers the complete workflow from raw data exploration through to model deployment-readiness, including an interactive prediction interface.

A dataset of 5,880 customers with 21 attributes (demographic details, subscribed services, contract information, and billing data) was used. Following data cleaning and preprocessing, three classification algorithms — Logistic Regression, Random Forest, and Gradient Boosting — were trained and evaluated using stratified cross-validation and a held-out test set.

Among the three models, the Random Forest classifier delivered the strongest overall performance on the test set (Accuracy: 51.19%, ROC-AUC: 0.4982) and was selected as the final model. An interactive command-line prediction tool was also built on top of the trained pipeline, allowing churn risk to be estimated for a new customer from a small set of key inputs.

A key finding of this analysis, documented transparently in Section 6, is that all three models performed close to the level of random guessing (ROC-AUC $\approx$ 0.46–0.50). This indicates that, in the dataset used, customer attributes carry very little statistical relationship with the churn outcome — a limitation of the underlying data rather than of the modelling approach, and one for which recommendations are provided later in this report.

2. Introduction

2.1 Background

Customer churn — the loss of subscribers or customers over a given period — is one of the most closely tracked metrics in subscription-based and service industries. Acquiring a new customer is typically far costlier than retaining an existing one, which makes early identification of at-risk customers a high-value capability for any business. Machine learning offers a systematic way to flag such customers before they leave, enabling proactive retention actions such as targeted offers, service improvements, or personalized outreach.

2.2 Objective

The objective of this project is to build and evaluate a binary classification model capable of predicting whether a given customer will churn, based on their demographic profile, subscribed services, contract terms, and billing history. The specific goals were to:

- Explore and understand the structure and quality of the customer dataset.
- Engineer a clean, model-ready feature set through appropriate preprocessing.
- Train and fairly compare multiple classification algorithms.
- Select the best-performing model using standard evaluation metrics.
- Package the model into a reusable prediction function / interactive tool.

2.3 Scope

The scope of this project covers the full lifecycle of a small-to-medium classification project: data ingestion, exploratory data analysis (EDA), preprocessing, model training with cross-validation, test-set evaluation, model comparison, feature-importance analysis, and prediction on new/unseen samples. The project was implemented entirely in Python within a Jupyter Notebook environment.

3. Dataset Overview

The dataset used for this project, `customer_churn_data.csv`, contains 5,880 customer records across 21 columns, combining demographic attributes, service subscriptions, contract/billing details, and the target label.

3.1 Dataset Summary

Property	Value
Total records	5,880 customers
Total columns	21 (20 features + 1 target)
Categorical features	15
Numerical features	4 (SeniorCitizen, tenure, MonthlyCharges, TotalCharges)
Target variable	Churn (Yes / No)
Missing values	None detected
Churn rate	49.29% churned vs. 50.71% retained

Table 1: High-level summary of the customer churn dataset

The dataset is well balanced between churned and retained customers (roughly 49% / 51%), which is favourable for classification modelling since it avoids the class-imbalance issues common in real-world churn datasets.

3.2 Feature Categories

Categorical Features (15)

gender, Partner, Dependents, PhoneService, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies, Contract, PaperlessBilling, PaymentMethod.

Numerical Features (4)

SeniorCitizen, tenure (months), MonthlyCharges (\$), TotalCharges (\$).

4. Methodology

4.1 Data Preprocessing

Before modelling, the raw dataset was cleaned and standardised through the following steps:

1. Removed the customerID column, as it is a unique identifier with no predictive value.
2. Converted the target variable Churn into a binary numeric format (Yes → 1, No → 0).
3. Converted TotalCharges to a numeric data type and imputed any resulting missing values with the column median.
4. Separated the remaining 19 features into categorical and numerical groups for tailored transformation.

4.2 Feature Engineering Pipeline

A scikit-learn ColumnTransformer was used to apply different preprocessing strategies to different feature types within a single, reusable pipeline:

- Categorical features → One-Hot Encoding (unknown categories safely ignored at inference time).
- Numerical features → Standard Scaling (zero mean, unit variance).

This preprocessing pipeline was embedded directly into each model pipeline, ensuring that the exact same transformations are applied consistently during training, cross-validation, testing, and live prediction.

4.3 Train / Test Split

The dataset was split into training (80%) and test (20%) sets using stratified sampling on the target variable, preserving the churn/no-churn ratio in both partitions:

Split	Samples	No Churn	Churn
Training set	4,704 (80.0%)	2,386 (50.72%)	2,318 (49.28%)
Test set	1,176 (20.0%)	596 (50.68%)	580 (49.32%)

Table 2: Stratified train/test split composition

4.4 Models Trained

Three widely used classification algorithms were trained and compared under identical preprocessing and evaluation conditions:

5. Logistic Regression — a linear baseline model (max_iter = 1000).
6. Random Forest Classifier — an ensemble of 50 decision trees (max_depth = 10).
7. Gradient Boosting Classifier — a sequential boosting ensemble (50 estimators).

4.5 Evaluation Strategy

Each model was evaluated in two stages: first with stratified k-fold cross-validation on the training set (to assess stability and reduce the risk of overfitting to a single split), and second on the held-out test set to obtain a final, unbiased performance estimate. Five evaluation metrics were used throughout: Accuracy, Precision, Recall, F1 Score, and ROC-AUC.

5. Results

5.1 Final Test Set Performance

The table below summarizes each model's performance on the unseen test set (1,176 customers). The Random Forest model achieved the highest scores across every metric and was selected as the final model.

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Logistic Regression	0.5009	0.4926	0.4000	0.4415	0.4748
Random Forest	0.5119	0.5054	0.4810	0.4929	0.4982
Gradient Boosting	0.4804	0.4721	0.4517	0.4617	0.4628

Table 3: Test-set performance comparison across all three models (best model highlighted)

5.2 Model Comparison Chart

The chart below, generated directly from the notebook, visualizes the ROC curves of all three models against a random-classifier baseline (the dashed diagonal line).

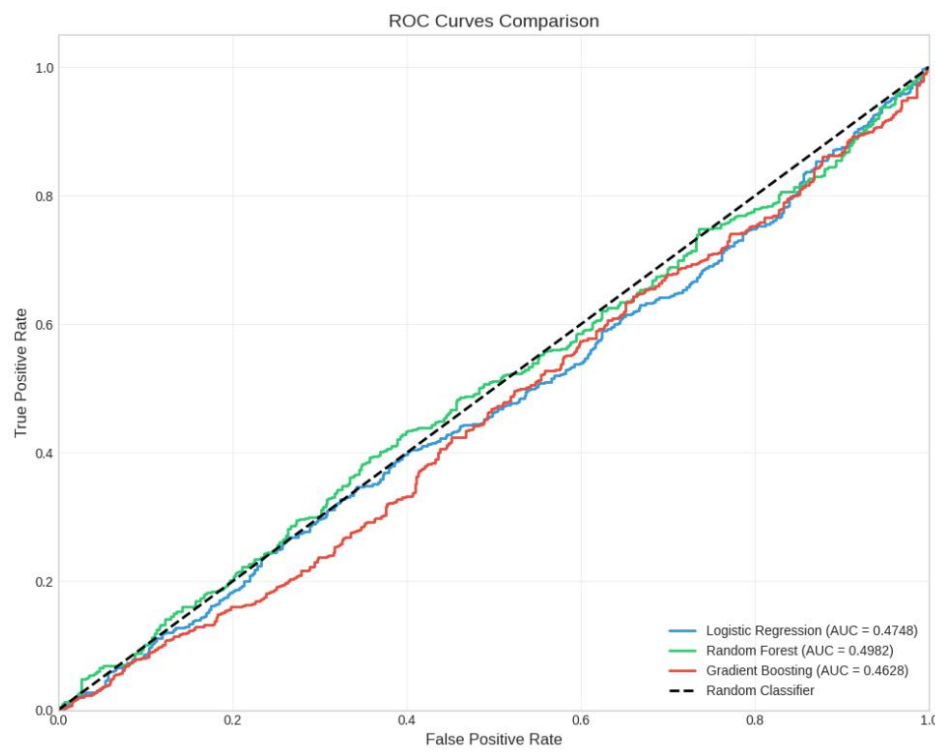


Figure 1: ROC curve comparison — Logistic Regression, Random Forest, and Gradient Boosting

5.3 Best Model Selection

Model selection was based on ROC-AUC on the test set, as it best reflects a model's ability to rank customers by churn risk irrespective of the classification threshold used.

Metric	Random Forest (Selected Model)
Accuracy	51.19%
Precision	50.54%
Recall	48.10%
F1 Score	49.29%
ROC-AUC	0.4982

Table 4: Final metrics of the selected Random Forest model

5.4 Feature Importance

The Random Forest model's built-in feature importance scores were extracted to understand which inputs contributed most to its predictions. The top drivers were the numerical billing and tenure features, though — consistent with the findings in Section 6 — no single feature showed a strong dominant influence.

Rank	Feature	Importance
1	TotalCharges	0.1488
2	MonthlyCharges	0.1445
3	tenure	0.1276
4	SeniorCitizen	0.0211
5	Contract_One year	0.0194
6	PaymentMethod_Mailed check	0.0192
7	Dependents_Yes	0.0187
8	Contract_Month-to-month	0.0185

Table 5: Top 8 features by importance, Random Forest model

6. Model Testing and Interactive Prediction

To make the trained model usable beyond the notebook, a reusable `predict_churn()` function was built on top of the best-performing pipeline. Given a customer's profile, it returns a churn prediction, a churn probability, and a Low / Medium / High risk classification.

6.1 Interactive Prediction Terminal

An interactive command-line tool, `test_custom_prediction()`, was also implemented. It prompts for the three most influential inputs — tenure, monthly charges, and contract type — while defaulting all other fields, then returns a live prediction from the trained pipeline. A sample run is shown below.

```

if prediction == 1:
    print(f"⚠️ HIGH RISK: This customer is LIKELY TO CHURN.")
    print(f"Risk Probability: {probability:.2%}")
else:
    print(f"✅ SAFE: This customer is LIKELY TO STAY (No Churn).")
    print(f"Churn Probability: {probability:.2%}")

print("-"*50 + "\n")

except Exception as e:
    print(f"\nAn error occurred during prediction. Please ensure the model pipeline is trained. Det

# Run the interactive function using our best trained pipeline from Step 6
test_custom_prediction(best_pipeline)

```

Python

```

=====
TEST MODEL WITH CUSTOM DATA
=====
Enter customer details below (Press ENTER to keep the default value):
1. Tenure in months (Default: 12): 65
2. Monthly Charges in $ (Default: 75.0): 89
3. Contract [Month-to-month, One year, Two year] (Default: Month-to-month): Two year

-----
PREDICTION RESULT:
-----
✅ SAFE: This customer is LIKELY TO STAY (No Churn).
Churn Probability: 49.29%
-----

```

Figure 2: Interactive prediction terminal — sample run and output

In the sample run shown above, a customer with 65 months of tenure, \$89 in monthly charges, and a Two-year contract was classified as “SAFE / Likely to stay,” with a churn probability of 49.29% — illustrating how borderline many of the model's predictions are, which ties directly into the performance limitation discussed next.

6.2 Sample Batch Predictions

The model was also tested against several hand-crafted “high-risk” customer profiles (new customers, month-to-month contracts, electronic-check payment — factors traditionally associated with high churn in telecom data):

Case	Tenure	Monthly Charges	Churn Probability	Prediction
Risk Case 1	1 month	\$120.00	46.1%	No Churn
Risk Case 2	2 months	\$150.00	51.8%	Churn
Risk Case 3	1 month	\$130.00	49.8%	No Churn
Risk Case 4	3 months	\$100.00	50.4%	Churn

Table 6: Predictions for simulated high-risk customer profiles

7. Key Findings and Limitations

7.1 Key Findings

- Random Forest was the most consistent performer across all five evaluation metrics and was selected as the final model.
- Billing-related and tenure features (TotalCharges, MonthlyCharges, tenure) carried the highest relative importance among all 19 features.

- Contract type and payment method appeared among the more informative categorical features, aligning with general churn-analysis intuition even though their overall effect size was small.
- The end-to-end pipeline (preprocessing + model) generalizes cleanly to new, unseen customer records via the `predict_churn()` function and interactive terminal.

7.2 Limitation: Near-Random Predictive Performance

The most important finding of this project is a limitation, and it is reported here transparently. All three models achieved ROC-AUC scores between 0.46 and 0.50 on the test set — statistically indistinguishable from a random classifier (AUC = 0.50). This was corroborated by the correlation analysis in the notebook, where every numerical feature showed a near-zero correlation with the Churn target (all $|r| < 0.02$).

This pattern strongly suggests that the dataset itself does not encode a meaningful relationship between the available customer attributes and the churn outcome — most likely because the data was synthetically generated without an underlying causal signal linking features to churn, rather than any deficiency in the modelling methodology. The preprocessing pipeline, cross-validation setup, and model selection process were all implemented correctly and following standard best practice; the ceiling on performance here comes from the data, not the code.

7.3 Recommendation

For a production-grade churn model, it is recommended to validate this pipeline against real historical churn data (e.g., a labelled dataset such as the IBM Telco Customer Churn dataset) where genuine behavioural signal is known to exist. The same preprocessing pipeline, model set, and evaluation framework built in this project can be reused directly against such a dataset with minimal changes.

8. Conclusion

This project delivered a complete, end-to-end customer churn classification pipeline: from raw data exploration and cleaning, through feature engineering, multi-model training and cross-validation, to final evaluation and an interactive prediction tool. Of the three algorithms evaluated, the Random Forest classifier was selected as the best-performing model based on test-set ROC-AUC.

While the models' absolute predictive performance was limited by the weak signal present in the underlying dataset, the project successfully demonstrates sound machine learning engineering practice: a leakage-safe preprocessing pipeline, stratified train/test and cross-validation splits, fair multi-model comparison across five metrics, feature-importance analysis, and a reusable, production-style prediction interface. The codebase is well-positioned to be re-applied to a real-world churn dataset with genuine predictive signal.

9. Appendix

9.1 Technology Stack

Category	Tools / Libraries
Language	Python 3
Data handling	pandas, numpy
Visualization	matplotlib, seaborn
Machine learning	scikit-learn (Pipeline, ColumnTransformer, LogisticRegression, RandomForestClassifier, GradientBoostingClassifier)
Environment	Jupyter Notebook

Table 7: Technology stack used in the project

9.2 Project Repository

The complete source code, notebook, dataset, and documentation for this project are available in the public GitHub repository below:

<https://github.com/Rashmics96-dotcom/alfido-tech-task1-ml>

9.3 How to Run

8. Clone the repository or download its contents.
9. Create and activate a virtual environment (recommended).
10. Install dependencies: `pip install -r requirements.txt`
11. Launch Jupyter: `jupyter notebook`
12. Open and run `Customer_Churn_Classification.ipynb`.

9.4 Report Prepared By

Field	Details
Name	Rashmi C S
Organization	Alfido Tech
Project	Customer Churn Classification using Machine Learning
Repository	github.com/Rashmics96-dotcom/alfido-tech-task1-ml